

Progress Tracker

Legend: ☐ Mastered ☐ Needs practice ☐ Not learned

Updated at the end of every document (see CURRICULUM.md §4 for what each document covers).

#	Document	Theory	Coding	Debugging	Architecture	Production	Interview
01	Python & Engineering Foundations	☐	☐	☐	—	☐	☐
02	APIs, HTTP, JSON & Backend Basics	☐	☐	☐	—	☐	☐
03	LLM Basics	☐	☐	☐	—	☐	☐
04	OpenAI API (Project 1)	☐	☐	☐	—	☐	☐
05	LangChain Basics	☐	☐	☐	—	☐	☐
06	Tools & Function Calling	☐	☐	☐	—	☐	☐
07	AI Agents (Project 2)	☐	☐	☐	—	☐	☐
08	RAG	☐	☐	☐	—	☐	☐
--	Checkpoint: Async Python	☐	☐	—	—	—	—
09	LangGraph	☐	☐	☐	☐	☐	☐
10	Agent Workflows (Project 3)	☐	☐	☐	☐	☐	☐
11	Multi-Agent Systems (Project 4)	☐	☐	☐	☐	☐	☐
12	Production AI Engineering	☐	☐	☐	☐	☐	☐
13	Testing, Evaluation &	☐	☐	☐	—	☐	☐

#	Document	Theory	Coding	Debugging	Architecture	Production	Interview
	Watching Your System						
14	Debugging Lab	—	—	☐	—	—	☐
15	Five Projects (index)	—	—	—	—	—	—
16	System Design & Architecture	☐	—	—	☐	☐	☐
17	Interview Preparation	—	—	—	—	—	☐
18	Capstone (Project 5)	☐	☐	☐	☐	☐	☐
19	MLOps & LLMOps	☐	☐	☐	☐	☐	☐

Projects

#	Project	Agents	Status
1	SupportDesk: Concierge & Triage	2	☐ Not started
2	ResearchHand: Worker & Verifier	2	☐ Not started
3	DocuMind: Retriever, Reasoner & Approval	3	☐ Not started
4	ContentForge: Supervisor-Led Team	5	☐ Not started
5	ContentForge Pro: Production Platform	5	☐ Not started
6 (bonus)	CodeGuard: PR-Review Team	4	☐ Not started
7 (bonus)	MCPForge: MCP Server	—	☐ Not started
8 (bonus)	MCPBridge: MCP Client Agent	1 (dynamic)	☐ Not started

Log

- 2026-08-27 — Curriculum created (`CURRICULUM.md`). No teaching started yet.
- 2026-08-27 — Added self-contained "Core Concepts" (What/Why/When/How, no external reading required) to every document 01-18 and the async prereq gate. Added Document 19 (MLOps & LLMOps), built on request, covering prompt/config versioning, eval-gated deploys, canary/blue-green/shadow deployment, drift monitoring, rollback. Added portfolio-pitch +

problems/solutions sections to all 5 project READMEs. Docker significantly expanded within Doc12 per request.

- 2026-08-27 — Restructured all 5 projects to be genuine multi-agent systems, agent count growing $2 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 5$ across the arc (per user request, reconciling with the "simple $\rightarrow$ agent $\rightarrow$ multi-agent" build methodology). Every project now has: a proper portfolio title (SupportDesk / ResearchHand / DocuMind / ContentForge / ContentForge Pro), a Type/Stack/Level/Tagline header, and an explicit "How To Build This — Step by Step" guide (each with a "what this step does" + "what changes vs. previous step" callout, sub-steps, reading-material references into each doc's Core Concepts, and an incremental project-structure tree). Synced new names/agent-counts into CURRICULUM.md §2, PROGRESS.md, and Doc15's index.
- 2026-08-27 — Fixed a PDF bug: document metadata Title was showing the file path instead of the real heading (script now extracts the markdown's H1). Updated all project + step titles to a descriptive, kebab-case portfolio style (e.g. "SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint") that states the deliverable, not just the topic. Split steps that bundled multiple agents/concerns into smaller, single-purpose steps: Project 2 now 4 steps (was 3), Project 3 now 5 steps (was 3), Project 4 now 5 steps (was 3), Project 5 now 4 steps (was 3) — each project's agents/concerns are now built and tested one at a time. Added an explicit "Final Deliverable" line after each project's last step, restating the full title and what it does. Per user instruction, no longer zipping/sending files each update — user syncs via their own `copy_to_local.sh`.
- 2026-08-27 — Added Project 6 (bonus, not in the numbered doc sequence): **CodeGuard**, a 4-agent multi-agent PR-review system (Security/Style/Test-Coverage reviewers + Supervisor), reusing every skill from Docs 01-19 in a different domain (dev tooling vs. content publishing) for portfolio variety, with deliberately less step-by-step hand-holding as an independence test. Added a "Setup (do this once, before Step 1)" section with exact `pip install / .env` commands to every project (1-6) — closing a real gap for jumping straight into coding without reading docs first. Fixed two PDF rendering bugs: (1) markdown tables immediately following a bold intro line with no blank line were rendering as raw pipe-separated text instead of an actual table, across 19 files; (2) the new per-step "Project: X — Step N of M" tag was merging into the following paragraph with no line break, across all 26 steps. Added the per-step project-title tag itself (`*Project: **Name** – Step N of M*`) to every step in every project per request. Investigated a user report of upside-down PDF text in VSCode's PDF viewer — verified via two independent tools (page rotation metadata, ghostscript render) that the PDF files themselves are structurally correct; likely a transient viewer-side scroll/render glitch, not a file defect. Enriched every step's "Project: X — Step N of M" tag with that step's own title, so all 26 tags are distinct instead of only the step number changing.
- 2026-08-27 — Rewrote every document (01-19 + async checkpoint) and every project (1-6), plus CURRICULUM.md, in simple, easy English — shorter sentences, plain words instead of technical/academic phrasing, per user request ("mushkil english words na hon, har jagah"). Structure, technical terms that must stay (Python/API/LangGraph names), and all content stayed the same — only the language got simpler.

- 2026-09-09 — Fixed the PDF header/footer per request: removed the Chrome-generated file-path footer line (e.g. "file:///tmp/.../pdf_html... 1/5"), but kept a real header showing the document's own title. Command-line Chrome flags only offer all-or-nothing header/footer control, so switched the PDF generation script from raw `google-chrome-stable --print-to-pdf` to Playwright (driving the same installed Chrome binary), which allows a custom header template (title only) and footer template (page number only, e.g. "2 / 7"). Regenerated all 28 PDFs and verified visually.
- 2026-09-09 — Added a "Setup for this document's practice code" + "How to run each exercise" block to all 14 documents with real hands-on coding (Doc01, 02, 04-13, 19) — exact `pip install` line and file-naming/run convention (`practice_basic.py` , etc.), so practice can start immediately without hunting for setup steps. Added short "why no code / where the code lives instead" notes to Doc03 (conceptual) and Doc14 (live, code lives in Projects 1-4). Standardized every document's Practice Exercises to exactly 5 scenarios (was as few as 3, as many as 6, inconsistently) — added Real-world/Edge-case scenarios to Doc01/03/08b, merged a 6th scenario into Failure on Doc04/06/07/08/09/11/12 — so every document now practices its reading material the same, predictable way. Regenerated all 28 PDFs.
- 2026-09-09 — Rewrote every one of the 5 practice scenarios in all 15 code-practice documents with a consistent What / Why / "When you'll hit this for real" (a concrete, memorable real-world trigger scenario) / How-to-code-it structure, per request — so each scenario is now explained well enough to recognize and recall when a similar real situation comes up later, not just a one-line task description. Added a "□ You Can Now Build Project N" pointer section, right before "Move On When," to the 5 documents where a project's prerequisites complete (Doc04 → Project 1, Doc07 → Project 2, Doc10 → Project 3, Doc11 → Project 4, Doc19 → Project 5), directing to that project's own Setup section as the actual build guide to follow. Regenerated all 28 PDFs.
- 2026-09-09 — Extended the same enrichment to the projects and remaining documents: added a "When you'll hit this for real" line to every one of the 26 steps across all 6 projects (right after each step's "What this step does"), and applied the What/Why/When/How-to structure to Doc16 and Doc17's practice exercises too (kept at 3 scenarios there, not 5, since those two documents' real practice is live design/interview review with me, not solo scripts — noted why explicitly). This completes the enrichment across every document and every project. Regenerated all 28 PDFs.
- 2026-09-09 — Built a real Hint/Solution system for Doc01, per request: every practice exercise and the Build Task (7 items total) now has its own `hints_and_solutions/` folder with 4 separate, progressively-revealing hint documents (concept → direction → pseudocode → small snippet) plus a solution document showing 2 different working approaches side by side, not just one — 35 new files, linked directly from the exercise in `01_python_foundations/README.md` . Fixed a real PDF-linking bug found along the way: links between documents were resolving to an absolute path inside the temporary build location instead of the real file, so clicking one opened nothing real — fixed by generating each intermediate HTML file next to its real `.md` file instead of in a scratch folder, so Chrome bakes in the correct real path when converting to PDF; verified the fix by inspecting the actual link target embedded in the PDF. This is Doc01 only so far — the same treatment across the other 19 documents and 6 projects is a large amount of remaining work (Doc01 alone was 35 files) and hasn't been done yet.

- 2026-09-09 — Built a permanent, reusable version of the PDF build tooling in `learning_langgraph/tools/`: `build_pdfs.py` (rebuild everything, or just specific files passed as arguments) and `watch_pdfs.py` (per request — stays running, checks every 2 seconds for any `.md` file newer than its `.pdf`, and rebuilds automatically). This replaces the one-off scratchpad script used earlier in the session, which didn't persist between sessions. From now on, editing any `.md` file and running `python3 tools/watch_pdfs.py` (left running in a terminal) keeps every PDF in sync automatically, without needing to ask for a manual rebuild.
- 2026-09-09 — Consolidated Doc01's Hint/Solution system per request, to cut the PDF count: instead of 5 separate files per exercise (Hint 1-4 + Solution = 35 files total), each exercise now has **one** combined file (`hints_and_solutions/<exercise>.md`) with all 4 hints and the solution inside it, each on its own page via a CSS page-break — so opening one PDF and paging through gives the same progressive reveal as before, just as one document instead of five. Added `.page-break` CSS support to `build_pdfs.py` for this. Also caught and filled a real gap: "Failure handling" (Doc01's 5th core practice scenario) had been skipped during the earlier What/Why/When enrichment pass and had no hints/solution — fixed now, brought in line with the other 6 items. Doc01 is now 8 hint/solution files total (was 35), all verified to open the correct linked PDF. **This one-file-per-exercise pattern is now the standard for every future document and project's hints/solutions**, not just Doc01.
- 2026-09-09 — Reworked Doc01's Hint/Solution content per request, so every hint and solution now carries **two reading depths inside the same page**: a "Simple Version" (plain framing, basic code) followed by a page-break and an "Intermediate Version" (proper Python idioms — type hints, dataclasses, formatters) — English stays simple and easy to read on both pages; only the code's complexity changes. The Real-world and Build Task solutions show 2 full working approaches at each depth, not just 1, per request. Files were renamed to descriptive names instead of generic ones (e.g. `basic2` → `venv_setup`, `intermediate1` → `env_parsing`, `intermediate2` → `logger_levels`, `realworld` → `config_logging_wiring`, `edgecases` → `env_edge_cases`, `failurehandling` → `failure_handling`).
- 2026-09-09 — Cut Doc01's file count again after feedback that the per-hint file split still looked like too many documents in the file explorer: merged each exercise's 4 separate hint files back into **one** `<exercise>_hints.md` file, with a clickable jump-to table of contents at the top and each hint's Simple/Intermediate pair on its own page-broken pair of pages. The Build Task's hints and solution live in its existing single `build_task.md` the same way. Doc01 is now down to 15 hint/solution files total (was 35, then 8, now 15 — 2 files per exercise: one `_hints.md`, one `_solution.md` — plus 1 for the Build Task). Verified with `pypdf` that Chrome's print-to-pdf bakes each markdown heading into a real PDF **named destination**, so a README link like `basic1_hints.pdf#hint-2` opens the correct file *and* jumps straight to that exact page — confirmed for every hint anchor across all 7 exercises plus the Build Task's 5 anchors (Hint 1-4 + Solution). All 16 Doc01 PDFs rebuilt and verified. Doc01 is complete under this pattern; further documents/projects are on hold until the user has tested this one.
- 2026-09-09 — Fixed a real portability bug: Chrome's print-to-pdf was baking the *build server's own absolute path* (`file:///home/administrator/...`) into every internal PDF link, so links

broke once the user synced files to their local machine via `copy_to_local.sh` at a different path. `build_pdfs.py` now post-processes every generated PDF with `pypdf` to rewrite those into relative paths (e.g. `hints_and_solutions/basic1_hints.pdf#hint-1`), so the whole PDF set stays clickable no matter where the `learning_langgraph` folder is copied to. This fix lives in the shared build tool, so it applies to every future rebuild across all documents. Also diagnosed why VS Code's own built-in PDF preview doesn't follow links (it's a sandboxed pdf.js webview that blocks `file://` navigation) — not a bug in the PDFs; the user is installing the `vscode-pdf` extension to test.

- 2026-09-09 — Added a nav bar (Back to exercise + Hint 1-4 + Solution links) to the top of **every physical page** in every Doc01 hint/solution file and the Build Task doc, not just page 1 — so no matter which page you're reading, you can jump straight to any other hint, the solution, or back to the exercise, without paging back to the start. Also found and fixed a real formatting bug while investigating a "links not clickable in VS Code" report: markdown bullet lists immediately following a bold intro line with no blank line between them (a pattern used by every one of Doc01's 7 exercise intros plus 3 Build Task sections) were rendering as run-on prose text instead of real bullets — fixed by inserting the missing blank line in all 11 spots; confirmed the exact same class of bug the 2026-08-27 log fixed for tables, just for lists, previously undetected.
- 2026-09-09 — Found and fixed the actual root cause of "Back to the exercise" (and "Back to the Build Task") landing on page 1 instead of the exact page you clicked from: Chrome's print-to-pdf only creates a PDF named destination for a heading/anchor **id** if something *inside that same document* already links to it — a cross-document link alone (from a hint file back to the README) is not enough to make Chrome register the destination. Confirmed this with an isolated test (an id with no in-document reference produced zero named destinations; the same id referenced by an in-document link worked immediately). Fixed by (1) converting each of Doc01's 7 exercise intros from a bold paragraph into a real heading with an explicit anchor id (`### Basic – typed function + custom error { : #ex-basic1 }`, using the newly-added `attr_list` markdown extension), and (2) adding a "Jump to an exercise" quick-nav line near the top of `01_python_foundations/README.md` that links to all 7 exercises plus the Build Task — which is also what supplies the required in-document reference for each destination to exist at all. Along the way, found and fixed the same missing-destination bug on the CURRICULUM.md breadcrumb link (and a separate double-hyphen anchor typo that existed independently of that bug, in 2 places). Wrote a full automated link-integrity checker (walks every PDF in Doc01, resolves every internal `/URI` link's target file and named destination against what's actually inside the target PDF) and ran it after every fix — final result: **all 17 relevant PDFs, every single internal link verified to open the exact right file and jump to the exact right page**, not just spot-checked.
- 2026-09-09 — Added a "The Story" section to the top of Doc01's README, in plain narrative language (per request that even a layman should follow it), tying together why the document's 5 separate-sounding topics (types/errors, venv, `.env`, logging, Build Task) are actually one connected story: building the reusable config + logging foundation every later project in the curriculum starts from. Reviewed every Doc01 solution file — all 7 exercises plus the Build Task already end with an explicit "which one should you use, and why" comparison between the

Simple/Intermediate versions and between multiple solution approaches (added earlier in the session); confirmed this satisfies the "explain the difference and which is best" request already, without needing new content.

- 2026-09-09 — Found that the same "missing PDF named destination" bug discovered in Doc01 was actually present across the **entire curriculum**, not just Doc01: every document's own breadcrumb link back to `CURRICULUM.md`, every project's links into its prerequisite documents' Core Concepts/Build Task sections, and two of `CURRICULUM.md`'s own internal section links were all broken the same way (either a double-hyphen anchor typo, or a heading with no in-document reference pointing to it, or — for the 6 projects — a genuinely stale anchor string left over from an old heading name change). Fixed all of it in one pass: corrected 19 documents' `CURRICULUM.md` breadcrumbs to the real slugs, added a 19-document quick-nav line to `CURRICULUM.md` §4 and a full section TOC at its top, added a "Jump to: Core Concepts · Practice Exercises · Build Task" quick-nav line to all 16 documents that have those sections (which also supplies the in-document reference each destination needs to exist at all), and fixed 40 broken cross-references across the 6 project READMEs. Extended `tools/check_links.py` (promoted from a one-off Doc01 script to a permanent tool) to walk every PDF in the whole repo; final run confirmed **all 43 PDFs, every internal link verified valid** — target file exists and the named destination it jumps to actually exists.
- 2026-09-10 — Rolled Doc01's full pattern (Story section, per-exercise `hints_and_solutions/` with Simple/Intermediate dual-depth pages, nav bar on every page, "which solution is best and why" comparisons) out to **Documents 02-06**, per explicit request to complete the curriculum "isi tarah" (this exact way), one document at a time, with a hard stop after Doc06 pending user review. Doc02 (HTTP Client Wrapper — 5 exercises + Build Task, 11 files) and Doc05 (LangChain — 5 exercises + Build Task, 11 files) were already complete. Doc03 (LLM Fundamentals — 5 reasoning-based exercises, no Build Task by design, 10 files) had its content but no PDFs — built and verified. Doc06 (Tools & Function Calling — 5 exercises + Build Task, 11 files) was the same — built and verified. Doc04 (OpenAI API / Project 1) was left mid-way by a background agent that hit an API rate limit: 3 of 5 exercises were done but `streaming_replies_solution.md`, both files for `context_limit_error` and `auth_error_cost_compare`, and the entire `build_task.md` were missing — wrote all 6 remaining files (the Build Task shows 2 full approaches, including a `ChatSession` -class version, ending in a "which one should you use" comparison) and verified. Ran `check_links.py` across all of Docs 01-06 individually at the end: **all 6 documents, every link verified valid**. Docs 07-19 and all 6 projects are intentionally not started yet — background agents had been dispatched for all of them in parallel but were stopped mid-way per explicit instruction to hold at Doc06 until this batch is reviewed.
- 2026-09-10 — Did a full independent re-verification pass across Docs 01-06 per request, checking everything fresh rather than trusting the prior run: re-ran `check_links.py` on all 6 (still all valid), checked every `.md` against its `.pdf` for staleness (none — every PDF is up to date with its source), swept for the blank-line-before-list rendering bug across all 6 READMEs and found 2 real instances that had been missed (the "Go Deeper" section's intro line in both Doc05 and Doc06 was directly followed by its reading-list bullets with no blank line between

them) — fixed both and rebuilt. Verified every solution file across all 6 documents actually ends with an explicit "which one is better and why" comparison (found 3 files whose comparison used different wording than the literal phrase checked for —

`config_logging_wiring_solution.md` 's "Why Solution 2 matters,"
`venv_setup_solution.md` 's "why this exact sequence matters" for its non-competing terminal-command steps, and `induced_hallucination_solution.md` 's "Which framing should you actually carry forward?" — read all 3 directly and confirmed they already satisfy the requirement, just phrased differently). Confirmed no leftover temporary `.html` files anywhere in Docs 01-06. This batch (01-06) is now final.

- 2026-09-10 — Doc01's hints were consolidated from 4 hints to 2 richer hints per exercise (each now with 3 depth levels — Basic/Intermediate/Advanced, up from Simple/Intermediate — per request to read as a story a layman can follow while still reaching real depth), for 6 of its 7 exercises plus the Build Task (`config_logging_wiring` kept its original 4 hints, since it already used the same 3-depth structure and didn't need shrinking). This consolidation left stale `#hint-3 / #hint-4` links behind in several places that hadn't been updated to match the new 2-hint count: the README's "Stuck?" line for 5 exercises plus the Build Task, and the internal nav bar inside `venv_setup_solution.md` and `env_parsing_solution.md` (24 broken links total, caught by `check_links.py`). Fixed every one, rebuilt all 16 Doc01 PDFs, and reconfirmed **all 16 PDFs, every link valid** — this is what "headings should be there" was catching: a hint heading a link pointed at no longer existed after the consolidation.
- 2026-09-10 — Deepened Doc06's Core Concepts per explicit request to make the tool-calling mechanism maximally clear: added a concrete before/after tool-description example, the actual JSON payload that crosses the wire during a real tool call, working Pydantic-rejection code showing exactly where a bad argument gets caught, a real tool-description-overlap example with its fix, and a new "Seeing the whole thing three ways" section explaining the full round-trip via a plain phone-call analogy, a numbered narrative trace, and matching code for every step — three independent framings of the identical mechanism, so at least one should land for any reader. Caught and fixed a blank-line-before-list bug introduced while writing this new content.
- 2026-09-10 — Major upgrade to Doc01's Hint/Solution system, per explicit request: every hint and solution went from **2 depth levels (Simple/Intermediate) to 3 (Basic/Intermediate/Advanced)**, every runnable code block now has a traced-through, verbatim-accurate **"Expected output"** block right after it, every hint's Advanced section ends with an explicit **"Difference between Basic, Intermediate, and Advanced"** paragraph, and every solution now shows **multiple numbered approaches per depth** (1 approach at Basic, 2 at Intermediate, 3 at Advanced — using judgment, not padding, when a task didn't genuinely support 3 distinct approaches) ending in a **"Which one should you actually use, and why"** final recommendation. Built the `basic1` exercise by hand first as the exact ground-truth template, then dispatched 4 parallel background agents (each briefed with that template plus a critical technical gotcha: python-markdown's `toc` extension auto-generates heading ids from heading TEXT, so adding a descriptive subtitle to `## Hint N` changes its id and silently breaks every existing link — fixed by pinning ids explicitly with `attr_list` syntax, `## Hint N – subtitle { : #hint-N }`) to apply the identical pattern to the remaining 6 exercises

(`venv_setup` , `env_parsing` , `logger_levels` , `config_logging_wiring` , `env_edge_cases` , `failure_handling`) and the Build Task. After all 4 agents finished, ran an independent full re-verification pass myself rather than trusting their self-reports: found and fixed 15 more instances of the blank-line-before-list bug across 8 files, and found 4 hint files (`venv_setup` , `env_parsing` , `logger_levels` , `config_logging_wiring`) each missing exactly one "Difference" callout (Hint 2's Advanced section specifically, in all 4 — a systematic miss, not random) — wrote and added the missing callout to each. Final state, verified fresh: all 16 Doc01 PDFs rebuilt, `check_links.py` reports ALL LINKS VALID, every hints file has exactly 4 "Difference" callouts, every solution file has its final recommendation. Doc01 is complete under this new 3-depth pattern and ready for user testing; Docs 02-19 and the projects still use the older 2-depth pattern and have not yet been upgraded to match.

- 2026-09-11 — **The full curriculum is now complete.** Rolled Doc01's 2-hint/3-depth (Basic/Intermediate/Advanced) Hint/Solution pattern out to every remaining document (02-19, plus the 08b async checkpoint) and every project (1-6), mostly via parallel background agents working from a shared reference template (`01_python_foundations/hints_and_solutions/env_parsing_*.md`), each instructed to verify its own work with `build_pdfs.py` + `check_links.py` before reporting done. Several documents/projects turned out to be genuinely incomplete under the old pattern too (missing exercises, missing solutions, or even a missing Build Task entirely) — Doc07, Doc09, Doc11, Doc18, Doc19, Project 2, and Project 3 all needed real content built from scratch, not just restructuring. Also substantially expanded Core Concepts topic coverage on Docs 06-11 per request (parallel tool calls, `tool_choice` , agent memory/planning, hybrid search, reranking, multi-agent communication patterns, and more — 4 new topics per document). Doc15 (previously a pure reference index with no practice exercise) got one added — running the curriculum's own 7-step planning process on a worked mini-scenario — after the user caught that it told readers to "write the 7 steps yourself" with nothing to actually practice against. Doc17's "Question Coverage" section was expanded from an abstract formula into a real worked question bank (5 Basic + 5 Intermediate + 5 Advanced + 3 scenario + 2 system-design questions) for 3 representative documents. Project 1 also got a "Before you read further — think about it yourself" callout added right after its Story section (two genuine guiding questions) and a pass simplifying ~15 overly-complex sentences — this "think first" pattern and the plain-English pass should be applied to the rest going forward, per explicit request.
- 2026-09-11 — Added **MCP (Model Context Protocol)** coverage per explicit request: 4 new Core Concepts topics in Doc06 (what MCP is and the problem it solves, its 3 building blocks — Tools/Resources/Prompts, servers/clients/ transports with a real working code example, and why it matters for multi-agent systems), plus a deep What/Why/How topic in Doc08 connecting a RAG retriever to MCP (wrapping `retrieve()` as an MCP Tool). Built two brand-new bonus projects from scratch, matching the existing projects' full structure and portfolio-quality bar: **Project 7 — MCPForge** (a real local MCP server exposing 2 Tools, 1 Resource, and 1 Prompt, backed by real SQLite persistence, built in 4 steps from a minimal stdio server up to a full discovery-driven client) and **Project 8 — MCPBridge** (a ReAct agent whose tools are discovered at runtime from live MCP servers rather than hardcoded, built in 4 steps from single-

server discovery up to multi-server namespacing and graceful failure handling when a server drops mid-task). Both fully verified.

- 2026-09-11 — Created the repo's top-level `README.md` (didn't exist before) for GitHub — title, one-line description, a table of all 19 documents with what each feeds into, a table of all 8 projects (agents/docs/what each proves), a "how to use this" quick-start, and PDF build instructions. Updated `CURRICULUM.md`'s Project ↔ Document map and `15_five_projects_index/README.md`'s Project Index to include Projects 5, 6, 7, and 8 (5 and 6 were missing from `CURRICULUM.md`'s table entirely). Ran a final full-repo `check_links.py` sweep with no arguments (every PDF in the whole repository, not just one folder at a time): **300 PDFs checked, every single internal link valid**. Every document (01-19 + the async checkpoint) and every project (1-8) is now complete, cross-linked correctly, and verified.